

# HƯỚNG DẪN CẤU TRÚC CODE SPRING BOOT CHO INTERN

*Controller - Service (Interface/Impl) - Repository - Mapper - DTO - Entity - Utils/Validator*

|           |                                                                                                     |
|-----------|-----------------------------------------------------------------------------------------------------|
| Mục đích  | Chuẩn hóa cách intern viết code, đặt tên file, chia layer và đi đúng luồng xử lý.                   |
| Phạm vi   | Áp dụng cho các module CRUD và các nghiệp vụ backend thông thường trong project Spring Boot.        |
| Cách dùng | Intern đọc tài liệu này trước khi code; reviewer dùng phần checklist ở cuối để review pull request. |

## Nguyên tắc

- Controller không chứa business logic.
- Service là nơi xử lý nghiệp vụ và điều phối các thành phần khác.
- Repository chỉ dùng để truy cập dữ liệu.
- Mapper chỉ dùng để convert giữa Entity và DTO.
- DTO phải tách rõ CreateRequest, UpdateRequest và Response.
- Utils/Validator chỉ xử lý phần dùng chung hoặc rule tái sử dụng; không biến thành nơi nhét mọi thứ.

## 1. Kiến trúc tổng quát

Luồng xử lý chuẩn nên được thống nhất cho tất cả intern để tránh viết code lệch trách nhiệm giữa các layer.

Client Request → Controller → Service → Validator / Utils → Repository → Mapper → Response

## 2. Vai trò của từng layer

| Layer             | Trách nhiệm                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Controller        | Nhận request, validate đầu vào bằng annotation, gọi service, trả response. Không xử lý nghiệp vụ phức tạp và không gọi repository trực tiếp. |
| Service           | Chứa business logic, điều phối validator, repository, mapper và các thành phần khác. Đây là layer quan trọng nhất.                           |
| Repository        | Truy cập cơ sở dữ liệu, khai báo query, không chứa business logic.                                                                           |
| Mapper            | Convert giữa Entity và DTO. Không query database và không validate nghiệp vụ.                                                                |
| DTO               | Tách rõ đối tượng nhận vào và trả ra để kiểm soát dữ liệu expose và dữ liệu được phép cập nhật.                                              |
| Entity            | Biểu diễn dữ liệu lưu trữ trong database, mapping với bảng/collection.                                                                       |
| Utils / Validator | Xử lý logic dùng chung, validate rule tái sử dụng, format dữ liệu, generate mã, parse giá trị, check nghiệp vụ lặp lại.                      |

## 3. Cấu trúc package nên dùng

Nên thống nhất theo package structure hiện tại của project để intern dễ bám theo và tránh tạo thêm nhiều biến thể. Với cấu trúc này, các package được chia theo vai trò kỹ thuật dùng chung cho toàn hệ thống.

src/main/java/com/yourcompany/project

```
├── configuration
├── controller
├── exception
├── mapper
├── model
│   ├── entity
│   └── dto
├── repository
├── service
│   ├── UserService.java
│   └── UserServiceImpl.java
├── common
│   ├── constant
│   ├── util
│   └── validator
└── Application.java
```

## 4. Quy ước đặt tên file

Intern cần tuân thủ quy ước đặt tên thống nhất để reviewer nhìn vào là hiểu ngay vai trò của từng class.

| Loại file         | Ví dụ với module User                          | Ghi chú                                                        |
|-------------------|------------------------------------------------|----------------------------------------------------------------|
| Controller        | UserController.java                            | Một module thường chỉ có một controller chính cho resource đó. |
| Service Interface | UserService.java                               | Chỉ khai báo method.                                           |
| Service Impl      | UserServiceImpl.java                           | Chứa business logic.                                           |
| Repository        | UserRepository.java                            | Kế thừa JpaRepository hoặc interface tương ứng.                |
| Mapper            | UserMapper.java                                | Convert Entity và DTO.                                         |
| Request DTO       | UserCreateRequest.java, UserUpdateRequest.java | Tách riêng create và update.                                   |
| Response DTO      | UserResponse.java, UserDetailResponse.java     | Không trả entity trực tiếp ra ngoài.                           |
| Entity            | User.java                                      | Mapping với database.                                          |
| Validator         | UserValidator.java                             | Validate nghiệp vụ tái sử dụng.                                |

## 5. Quy tắc bắt buộc theo từng layer

### Controller

- Chỉ nhận request, gọi service và trả response.
- Không gọi repository trực tiếp.
- Không nhét business logic vào controller.
- Không convert entity thủ công nếu mapper đã chịu trách nhiệm đó.

### Service

- Mọi business logic phải nằm ở đây hoặc ở domain/service hỗ trợ.
- Interface chỉ khai báo method; impl mới xử lý logic.
- Dùng @Transactional phù hợp cho create/update/delete hoặc logic cần transaction.

### Repository

- Chỉ viết query và thao tác dữ liệu.
- Không xử lý rule nghiệp vụ.
- Không lạm dụng native query khi JPA hoặc projection đã đủ.

### Mapper

- Chỉ convert dữ liệu.
- Không query database trong mapper.
- Không validate nghiệp vụ trong mapper.

### DTO

- Không dùng chung một DTO cho create, update và response.
- Field nào client không được sửa thì không đưa vào request DTO.
- Response chỉ expose dữ liệu cần thiết.

## Utils / Validator

- Utils chỉ xử lý helper dùng chung, không gắn chặt vào nghiệp vụ cụ thể.
- Validator nên xử lý rule tái sử dụng như kiểm tra trùng email, trạng thái hợp lệ, điều kiện được phép update.

## 6. Ví dụ module mẫu

Dưới đây là skeleton tối thiểu để intern nhìn theo và code đúng cấu trúc.

### UserController.java

```
@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @PostMapping
    public ApiResponse<UserResponse> create(@Valid @RequestBody UserCreateRequest request) {
        return ApiResponse.success(userService.createUser(request));
    }

    @PutMapping("/{id}")
    public ApiResponse<UserResponse> update(
        @PathVariable Long id,
        @Valid @RequestBody UserUpdateRequest request) {
        return ApiResponse.< UserResponse >builder()
            .result(result)
            .build();
    }
}
```

### UserService.java

```
public interface UserService {
    UserResponse createUser(UserCreateRequest request);
    UserResponse updateUser(Long id, UserUpdateRequest request);
    UserResponse getById(Long id);
}
```

### UserServiceImpl.java

```
@Service
@RequiredArgsConstructor
@Transactional
public class UserServiceImpl implements UserService {

    private final UserRepository userRepository;
    private final UserMapper userMapper;
    private final UserValidator userValidator;

    @Override
    public UserResponse createUser(UserCreateRequest request) {
        userValidator.validateCreate(request);
        User user = userMapper.toEntity(request);
        user = userRepository.save(user);
        return userMapper.toResponse(user);
    }
}
```

### UserRepository.java

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    boolean existsByEmail(String email);
}
```

## UserMapper.java

```
@Mapper(componentModel = "spring")
public interface UserMapper {

    User toEntity(UserCreateRequest request);

    UserResponse toResponse(User user);

    void updateEntity(@MappingTarget User user, UserUpdateRequest request);
}
```

## UserValidator.java

```
@Component
@RequiredArgsConstructor
public class UserValidator {

    private final UserRepository userRepository;

    public void validateCreate(UserCreateRequest request) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new BadRequestException("Email already exists");
        }
    }
}
```

## 7. Những lỗi hay mắc

- Gọi repository trực tiếp trong controller.
- Dùng một DTO cho tất cả create, update, response.
- Trả entity ra API thay vì response DTO.
- Đưa business logic vào mapper hoặc util.
- Tạo CommonUtils thành nơi nhét mọi thứ không biết để đâu.
- Đặt tên file không theo convention khiến code khó tìm.
- Dùng native query quá sớm khi chưa thật sự cần.
- Không tách validate annotation và validate nghiệp vụ.